

BoustoDoc: Implementation Details

Anonymous ACCV 2026 Submission

Paper ID #548

Abstract. Comprising implementation details for BoustoDoc, this supplementary document describes the shared data protocol, detector training recipe, OCR adaptation, routing policy, output serialization, and runtime configuration used in the submitted pipeline. It complements the main paper, which remains the primary source for the method, results, ablations, and error analysis. The pipeline detects document regions, routes each predicted type to its designated recognition branch, applies TrOCR to handwritten, printed, and annotation regions and Qwen3-VL to formula and table regions, and serializes schema-valid page outputs. The material is provided to make the experimental setup easier to inspect and reproduce, without introducing new results, corrected paper text, or post-deadline method changes.

1 Shared Data Protocol

1.1 Gold/Silver Data Usage

Gold and silver annotations serve distinct purposes. The human-annotated gold set defines the supervised training, validation, and internal-test protocol in Table 1. The silver set provides noisy auxiliary supervision for module-specific warm-up or augmentation, including OCR warm-up and rare-class detector augmentation. Silver data is excluded from checkpoint selection, hyperparameter tuning, and final metric reporting; all controlled evaluation remains tied to gold annotations.

1.2 Page-Level Internal Split

The gold split is performed at the page level: all regions from one page stay in the same subset, avoiding leakage between visually related crops. The protocol partitions 1,330 gold pages with seed 1089 into 931 training pages, 199 validation pages, and 200 internal-test pages, corresponding to a 70/15/15 split. Shuffling is stratified by source—archive, dictation, school, and university—so each subset preserves a comparable source mix. The validation subset is used for checkpoint selection and hyperparameter tuning; the internal test subset is reserved for final reporting because official test labels are hidden. Tables 1 and 2 summarize the resulting partitions.

Table 1: Internal page-level split used for controlled evaluation.

Split	Pages	Regions	Usage
Gold train	931	18,007	Supervised training
Gold validation	199	3,823	Checkpoint selection and tuning
Gold internal test	200	3,821	Final internal evaluation
Silver auxiliary train	8,207	161,065	Noisy auxiliary training only

Table 2: Source distribution of the internal gold split.

Source	Train	Validation	Test	Total
archive	89	19	19	127
dictation	251	54	54	359
school	478	102	102	682
university	113	24	25	162

1.3 General Preprocessing Rules

Detector labels keep the seven RUKOPYS region types, while OCR crops inherit the split assignment of their source page; no validation or internal-test crop is used for training. Before conversion or cropping, bounding boxes are validated, clamped to image boundaries, and filtered for positive area. Text-bearing regions preserve the original visible transcription target for training and serialization; evaluation normalization is applied only when computing metrics. Image and graph regions retain empty text because they are non-transcription regions.

2 Detector Training Details

2.1 Detector Training Data Preparation

The detector is fine-tuned from a DocLayout-YOLO document-layout checkpoint. Training examples are built from page images and region annotations in the seven RUKOPYS classes: `handwritten`, `printed`, `formula`, `table`, `annotation`, `image`, and `graph`. The final fine-tuning stage uses the curated merged detector corpus exported in YOLO-style `images` and `labels` folders. Silver data is not sampled directly during final fine-tuning; auxiliary samples are used through the earlier class-aware data-construction stage to increase exposure to rare region types.

2.2 Class Mapping and Bounding Box Conversion

Region annotations are converted from the source COCO-style bounding-box representation into normalized YOLO format. Each bounding box is clamped to image boundaries and discarded if it has invalid coordinates or non-positive area. The detector class mapping is fixed across training, validation, inference, and submission serialization.

Table 3: Detector class mapping.

YOLO ID	Region type
0	handwritten
1	printed
2	formula
3	table
4	annotation
5	image
6	graph

Table 4: Detector training hyperparameters.

Parameter	Value
Model	DocLayout-YOLO
Initialization	DocStructBench pretrained checkpoint
Number of classes	7
Batch size	8
Training image size	1024
Optimizer	Adam
Initial learning rate	0.001
Weight decay	0.0005
Momentum	0.9
V4.1 mosaic	0.1
V4.2 mosaic	0.25

059

2.3 Class-Aware Augmentation

059

060

061

062

063

064

065

066

067

068

Class-aware augmentation is used to reduce the effect of class imbalance. The data-construction stage assigns page-level priorities from region composition, groups classes into head, mid-frequency, and tail categories, and applies a repeat factor of up to 4 for pages containing rare or structurally important regions. The merged detector corpus contains 1,809 original images and 747 augmented images, with no boxes removed during preprocessing. This procedure increases the main underrepresented classes from 544 to 1,560 **image** instances, 91 to 238 **graph** instances, 980 to 1,778 **annotation** instances, and 674 to 1,106 **table** instances.

068

069

2.4 Detector Training Hyperparameters

069

070

071

072

073

074

075

Detector training and final submission inference deliberately use different image sizes. Training is performed with image size 1024 to reduce memory usage and improve throughput when multiple page images are processed in parallel. For the final detection pass used to generate the submission, the image size is increased to 1280 so that the detector can exploit additional fine-grained layout information at inference time.

2.5 Detector Inference and Ensemble

The final detector output is produced by an ensemble of two checkpoints, V4.1 and V4.2. Candidate boxes from both checkpoints are merged and then filtered during final post-processing. The submitted detector inference configuration uses image size 1280, confidence threshold 0.20, maximum detections per image 300, and class-agnostic NMS with IoU 0.55. In the internal 200-image COCOeval report with 3,821 ground-truth boxes, the V4.1+V4.2 ensemble reaches AP_{50} of 0.7022 and $AP_{50:95}$ of 0.4424, outperforming both individual checkpoints.

3 HPA OCR Training Details

3.1 HPA Crop Data Preparation

The handwritten, printed, and annotation (HPA) OCR branch operates exclusively on localized region crops extracted from the primary document pages. To prevent cross-region visual context leakage, all data partitioning is enforced strictly at the page level, individual text crops inherit the exact dataset split assignment of their source document. Bounding boxes are cropped directly using the ground-truth or predicted coordinates with zero padding applied to the horizontal or vertical axes. The image loading pipeline opens each crop file and standardizes the visual inputs by converting them into a three-channel RGB color space. Scale normalization and tensor formatting are managed directly by the image processing component of the text recognition processor, which resizes all input regions to a fixed dimension of 384×384 pixels before tokenization.

3.2 Tokenizer Adaptation

The underlying text recognition architecture is initialized from a pretrained Cyrillic handwritten text recognition baseline. Because the base model vocabulary lacks specific characters required for standard Ukrainian orthography, a tokenizer adaptation layer is executed prior to fine-tuning. The vocabulary is explicitly expanded with eight Ukrainian-specific Cyrillic characters: i , $\text{\u0456}$, $\text{\u0457}$, $\text{\u0435}$, $\text{\u0440}$, $\text{\u0418}$, $\text{\u0419}$, and $\text{\u0417}$. Following this vocabulary modification, the decoder’s token embedding weights are resized to match the new vocabulary dimension, and the model’s structural configuration parameters are updated. To ensure stable autoregressive decoding, the decoder start, padding, and end-of-sequence token identifiers are explicitly synchronized with the adapted tokenizer state.

3.3 Silver-to-Gold Curriculum

To mitigate data scarcity and robustly learn varying handwriting styles, training follows a structured three-phase curriculum. The model first undergoes a silver warm-up phase, training for 1.0 epoch on noisy auxiliary text crops with a learning rate of 3×10^{-5} to establish broad Cyrillic character alignment. In the second phase, the model is fine-tuned on high-quality, human-annotated gold crops for

Table 5: HPA OCR curriculum.

Stage	Data	Epochs	Learning rate
Silver warm-up	Silver HPA crops	1	3×10^{-5}
Gold fine-tuning	Gold HPA crops	6	1×10^{-5}
Gold recovery	Gold HPA crops	2	5×10^{-6}

Table 6: Type-specific decoding settings for HPA regions.

Region type	Branch	Decoding budget
handwritten	TrOCR HPA beam search	3, max new tokens 192
printed	TrOCR HPA beam search	3, max new tokens 128
annotation	TrOCR HPA beam search	1, max new tokens 16

114

115

116

117

118

119

120

6.0 epochs at a learning rate of 1×10^{-5} to capture exact domain features. Finally, a gold recovery phase is executed for 2.0 epochs at a reduced learning rate of 5×10^{-6} to stabilize the adapted weights. The optimization pipeline utilizes the Adafactor optimizer with a weight decay of 0.01 and a cosine learning rate scheduler preceded by a 5% linear warmup allocation. The maximum target sequence length is constrained to 192 tokens, and a uniform batch size of 32 is maintained across all devices.

114

115

116

117

118

119

120

121

121

3.4 Type-Specific Decoding

122

123

124

125

126

127

128

129

130

131

132

At inference time, text generation parameters are dynamically adjusted based on the predicted region type routed from the layout analysis stage. This per-class specialization prevents text over-generation on short annotations while allocating sufficient token budgets for long textual sequences. Handwritten regions are decoded using an auto-regressive beam search with a beam width of 3, a maximum allocation of 192 new tokens, and a standard length penalty of 1.0. Printed text regions share the beam width of 3 but are bounded by a moderate maximum budget of 128 new tokens. Conversely, annotation regions are processed using a greedy search variant with a beam width of 1, a maximum constraint of 16 new tokens, and a reduced length penalty of 0.8 to penalize excessive length expansion.

122

123

124

125

126

127

128

129

130

131

132

133

133

3.5 HPA Validation Protocol

134

135

136

137

138

139

Checkpoint monitoring and final model selection are governed by an internal validation protocol tied to the page-level stratified gold validation split. Rather than calculating intermediate validation performance under generic generation settings, the evaluation layer integrates a specialized checkpoint callback that decodes each validation crop according to its specific region-type budget. At the conclusion of each training epoch, this callback segregates the validation entries

134

135

136

137

138

139

by class, performs type-aware inference, and aggregates the resulting transcriptions. The selection criterion is defined by the lowest overall Character Error Rate computed across all valid handwritten, printed, and annotation validation regions. The checkpoint minimizing this metric is frozen and compiled as the final model weight for pipeline integration.

4 Qwen3-VL LoRA Training Details

4.1 Data Preparation

The Qwen branch is routed only to `formula` and `table` regions in the final pipeline, not to full pages. Its adapter is trained with a three-stage curriculum. Stage 1 performs a silver warm-up using silver instruction examples to establish the hybrid prompt behavior. Stage 2 initializes from the Stage 1 adapter and fine-tunes on gold training records with a held-out gold validation subset. Stage 3 initializes from the previous adapter and uses cached hard-type crop-level instruction data constructed from scorable text-bearing regions, including `handwritten`, `printed`, `formula`, `table`, and `annotation`; non-textual `image` and `graph` regions are excluded from this crop transcription set. Each crop inherits the split of its source page.

4.2 Instruction Data Construction

Instruction examples pair a crop image with a concise task prompt and the expected transcription. Formula targets copy the visible mathematical or chemical notation, using LaTeX when appropriate. Table targets preserve the visible row order and use pipe-separated rows. Targets do not include explanations, solved answers, inferred missing content, JSON wrappers, or Markdown fences.

4.3 LoRA Training Configuration

The adapted model is based on `Qwen3-VL-8B-Instruct`. We use a LoRA adapter with 4-bit NF4 model loading through `bitsandbytes`. Stage 3 continues from the previous LoRA adapter rather than initializing a new adapter structure, and trains it with supervised instruction examples for structured crop transcription.

4.4 Prompt Templates

The Qwen prompt is assembled as a source-aware hint, followed by a type-specific crop instruction and common output guardrails. The source hint only describes the document domain and does not provide target text. For formula crops, the instruction asks the model to read the visible mathematical, logical, physical, or chemical expression exactly, to return only the formula text, to preserve symbols, indices, superscripts, subscripts, arrows, fractions, matrix or determinant structure, punctuation, numbering, and correction markers, and not to solve,

Table 7: Qwen3-VL LoRA configuration.

Parameter	Value
Base model	Qwen3-VL-8B-Instruct
Adaptation	LoRA adapter with 4-bit NF4 loading
Quantization	4-bit NF4
Final routed types	formula, table
Decoding	Greedy decoding
Maximum new tokens	192
Formula target	Visible mathematical or chemical expression
Table target	Pipe-separated rows

Table 8: LoRA hyperparameters for the Qwen3-VL branch.

Parameter	Value
LoRA rank	32
LoRA alpha	64
LoRA dropout	0.05
Target modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Training precision	NF4 backbone, fp16 compute, fp32 LoRA params
Batch size	2 per device, gradient accumulation 8
Learning rate	8×10^{-6}

176
177
178
179
180
181
182
183

simplify, normalize, explain, or convert notation. For table crops, the instruction asks the model to return one pipe-separated line per visual row, preserve empty cells as empty fields, keep row and column order, preserve wrapped text, numbers, units, punctuation, dashes, and visible spelling mistakes, and not to infer missing cells, rebalance columns, summarize, or explain. Common guardrails require every output token to be supported by the crop and prohibit translation, grammar correction, spelling normalization, abbreviation expansion, hidden-text inference, JSON, Markdown, and explanatory text.

176
177
178
179
180
181
182
183

184

4.5 Qwen Inference Settings184

185
186
187
188
189
190
191
192

At inference time, the Qwen branch loads the selected Qwen3-VL base model and the stage-3 LoRA adapter with 4-bit NF4 quantization. It processes only routed formula and table crops, using a crop padding ratio of 0.04 and a maximum crop pixel budget of 262,144. Decoding uses greedy generation with one beam, no sampling, and a maximum of 192 new tokens. The final cleanup removes code fences and wrapper artifacts, while the prompt guardrails discourage JSON, Markdown, explanations, unsupported completion, translation, grammar correction, spelling normalization, and hidden-text inference.

185
186
187
188
189
190
191
192

Table 9: Routing policy used in the final pipeline.

Region type	Recognition branch
handwritten	TrOCR HPA
printed	TrOCR HPA
annotation	TrOCR HPA
formula	Qwen3-VL + LoRA
table	Qwen3-VL + LoRA
image	Empty text
graph	Empty text

5 Final Inference and Routing

At runtime, the input directory is resolved either from a `metadata.jsonl` file or by recursively discovering image files under the input image directory. When metadata is available, the pipeline preserves the metadata record list as the set of pages to process; otherwise, discovered image files are processed in sorted order. Each input record produces exactly one output row.

5.1 Region Routing Policy

At inference time, each page is processed by the detector ensemble. The predicted boxes are merged, filtered, clamped to image boundaries, deduplicated, and sorted in reading order. Each remaining region is routed according to its predicted type. HPA regions are sent to TrOCR, formula and table regions are sent to Qwen3-VL, and image or graph regions are preserved with empty text.

5.2 Post-Processing

Post-processing consists of candidate merging across detector checkpoints, class-agnostic NMS, coordinate clamping, duplicate removal, and reading-order sorting. The final inference configuration is summarized in Table 10.

5.3 Output Serialization and Validation

The final output is serialized as `submission.csv` with one row per input image and exactly two columns, `image` and `regions`. Each `regions` value is a JSON list sorted in reading order. Each region object contains exactly the keys `bbox`, `type`, and `text`; bounding boxes contain four values, and region types must belong to the seven valid RUKOPYS labels. The pipeline validates the row count, column set, JSON structure, region keys, bounding-box length, and region labels before writing the final CSV.

For reproducibility, the detector settings reported in this section correspond to the final submitted inference notebook. Detector training used image size 1024, whereas the submitted inference run used image size 1280 with a detector confidence threshold of 0.20.

Table 10: Final inference settings.

Parameter	Value
Detector checkpoints	V4.1 + V4.2 ensemble
Image size	1280
Confidence threshold	0.20
Max detections per image	300
NMS type	Class-agnostic NMS
NMS IoU	0.55
Deduplication IoU	1.01 (effectively disabled after NMS)
Qwen crop padding	0.04
Maximum crop pixels	262,144
Qwen crop max new tokens	192
Output sorting	Top-to-bottom, then left-to-right

Table 11: Software and runtime environment.

Item	Setting
Deep learning framework	PyTorch
OCR/VLM library	Transformers
LoRA library	PEFT
Quantization	bitsandbytes 4-bit NF4
Detector framework	DocLayout-YOLO / Ultralytics-compatible backend
Image processing	OpenCV / Pillow
Runtime mode	Offline inference
Qwen training hardware	Kaggle, 2×NVIDIA T4 GPUs
Qwen stage-3 training time	Approximately 13 hours
Full-system inference hardware	Kaggle, 2×NVIDIA T4 GPUs
Full-system inference time	Approximately 1 hour
Qwen training placement	Balanced device map, about 14 GiB per GPU
VLM utilities	qwen-vl-utils, TRL, datasets

6 Hardware, Runtime, and Reproducibility

The final system is run in an offline inference setting with all required model weights and processor artifacts stored locally. Detector, HPA, and Qwen runtime details are summarized in Table 11.

The final inference package contains the detector checkpoints, TroOCR model, Qwen3-VL base model, LoRA adapter, tokenizer or processor artifacts, inference script, dependency list, and output validation logic. Runtime inference does not require external network access.